GRAPH_ANOMALY

NORMAL TO ANOMALY

Narration Script & Visual Storyboard — Companion to the Interactive Explainer

38 scenes · 2,730 narration words · est. 18 – 26 minutes at documentary pacing

This is the full narration text and scene-by-scene visual spec behind the published interactive artifact **"Normal to Anomaly"** (built because this environment has no video-rendering pipeline — no ffmpeg, no video-export tool). The interactive version is the primary deliverable: it actually plays the animations and speaks the narration in-browser. This document is the same content in a portable, linear form — useful for reading end to end, or for handing to a video editor / voice narrator to produce a literal rendered video file elsewhere.

ACT 1 — INTRODUCTION

01 The Research Question

~25s

This is GRAPH_ANOMALY. The project asks a very specific question. Not simply: can we classify this subject as autistic or control. Instead: can we learn how brain connectivity subgraphs normally evolve over time, notice when they deviate from that learned normal, and deliberately generate meaningful deviations that can help when only a few real abnormal examples exist.

ON SCREEN

Brain silhouette forms; 26 ROI dots fade in. Split cards: steady teal sparkline (normal) vs. flickering violet one (anomalous).

---

02 Why Not a Direct Classifier?

~31s

ABIDE gives us one diagnosis label per subject — not a label on any specific brain event. A classifier trained on that would learn whatever incidentally correlates with site or age, not a real mechanism. So this project reframes the problem: train only on healthy dynamics, then measure deviation from what was learned. That deviation is mechanistically interpretable — it says exactly which subgraph, at which moment, stopped looking normal.

ON SCREEN

Two cards contrast a subject-to-label classifier (crossed out, red) against learn-normal-then-measure-deviation (teal, checked).

ACT 2 — THE DATA

03 ABIDE I: 871 Subjects, 20 Sites ~23s

The project uses ABIDE the First — the Autism Brain Imaging Data Exchange. Eight hundred seventy-one subjects, pooled from twenty different acquisition sites around the world, each contributing a resting-state functional MRI scan. Four hundred three of them carry an autism spectrum disorder diagnosis. Four hundred sixty-eight are typical controls. Watch them appear.

ON SCREEN

871 dots animate into a grid, then split into 403 amber (ASD) and 468 teal (control) groups; 20 site chips fade in below.

---

04 One Subject, 360 Regions ~29s

Zoom into one subject. Before anything else happens, that subject's brain has already been divided — by the dataset's own preprocessing — into 360 regions of interest, called ROIs, each covering a distinct patch of cortex. Every ROI produces its own signal over the course of the scan. The input, at this point, is not yet a graph. It is three hundred sixty separate time series.

ON SCREEN

Brain outline with 170 ROI dots fading in; zoom card '360 REGIONS'; five labeled sparklines (ROI 1, 2, 3, ■, 360) draw in.

---

05 From Two Signals to One Edge ~29s

Take any two ROIs. Correlate their time series with a Pearson correlation — how similarly do they rise and fall together. A strong positive correlation says these two regions tend to activate together. A negative correlation says they move in opposite directions — and this project keeps that sign, rather than only keeping the strength. That signed correlation number becomes the weight of an edge between them.

ON SCREEN

Two sparklines (ROI A, ROI B) draw in; Pearson-r equation appears; two nodes A/B connect with a thickening colored edge.

---

06 Sliding the Window: One Subject Becomes Many Graphs ~28s

A single correlation over the entire scan would hide exactly what this project needs — whether connectivity changes over time. So instead of one graph, the scan is cut into overlapping thirty-timepoint windows, sliding forward ten timepoints at a time. Every window gets its own functional connectivity matrix, and therefore its own graph. One subject becomes a sequence: G1, G2, G3, and onward.

ON SCREEN

A timeline scrubs with a moving 30-TR/10-TR-stride window; three small node-link graphs (G1, G2, G3) build in sequence.

---

07 Keeping Only the Strongest 20% ~28s

A complete 360-node graph has an enormous number of possible connections — most of them weak and probably spurious. So every window's matrix is sparsified: rank every connection by the absolute strength of its correlation, and keep only the strongest 20 percent. Weak edges disappear. Strong edges remain, still signed, still carrying their original value — only their magnitude decided whether they survived.

ON SCREEN

A dense random graph on ~22 nodes; edges rank by |weight| and the weakest 80% fade out, leaving a sparse colored graph.

---

08 Splitting Subjects, Not Windows ~30s

Before any model sees this data, all 871 subjects are locked into three groups: 609 for training, 131 for validation, 131 for test — stratified so the ASD-to-control ratio holds in each. This split happens at the subject level. Every window from one subject goes entirely into one group. If windows leaked across groups, a model could partly memorize that person's individual connectivity fingerprint instead of learning general dynamics.

ON SCREEN

871 dots regroup into three cards (Train 609 / Validation 131 / Test 131) with control/ASD counts; leakage do/don't note.

ACT 3 — SUBGRAPHS & GIN

09 One Seed, One Local Neighborhood ~25s

Feeding a full 360-node graph straight into a temporal model is both computationally unmanageable and impossible to interpret — 'the brain changed' is not an actionable finding. So the project picks a seed ROI and extracts its one-hop neighborhood: that region, plus everything directly connected to it. This much smaller local subgraph is what actually gets modeled.

ON SCREEN

Brain with 160 dots; one seed ROI highlighted; arrow into a star-shaped 1-hop neighborhood graph (seed + 8 neighbors).

---

10 Stable Tracking: Same Circuit, Changing Weights ~37s

How do you compare a subgraph across time if its membership keeps shifting? Two options were tested. Dynamic tracking lets neighborhood membership change every window — measured directly, that gave a mean overlap of only 0.378 between consecutive windows: barely the same graph. Stable tracking instead fixes ROI membership once, at a reference window, and only lets the edge weights evolve. That is what the project uses — because it lets a change in connectivity mean one unambiguous thing: this physical circuit behaved differently.

ON SCREEN

Split screen: dynamic tracking (A,B,C,D → A,B,E,F → A,G,H,F) vs. stable tracking (same 4 nodes, ring thickness changes).

---

11 Why Only 20 Seed ROIs? ~36s

The extraction method works for all 360 candidate ROIs as seeds — that is not the limitation. The limitation is compute: extracting all 360, for every subject, every epoch, measured at roughly 71 seconds per subject — about 12 hours for one pass over the 609 training subjects, repeated every epoch. So Phase 1 trains and evaluates on a fixed, deterministic, stratified subsample: 20 seed ROIs, chosen once with a fixed random seed, cached to disk. A compute-budget decision, documented as exactly that.

ON SCREEN

Grid of 360 ROI dots; 20 are highlighted amber as the deterministic seed subsample; compute-budget note below.

---

12 GIN: Watching a Node Listen to Its Neighbors ~39s

A subgraph is nodes and weighted edges — not something a recurrent network can consume directly. The weighted GIN turns it into one vector. Watch node C. Every neighbor — B, A, D, E — sends C a message, and that message is scaled by the real signed connectivity weight on that specific edge. C aggregates everything it receives, combines that with its own previous state, and updates. This happens for every node, twice — two GIN layers — so information has time to travel two hops through the neighborhood.

ON SCREEN

Star graph: node C at center, neighbors B/A/D/E send colored pulses along weighted edges into C; C recolors after aggregation.

---

13 The Actual GIN Update Rule ~40s

Written precisely, this is what happens at every node v, every layer. The neighbor sum uses the real weighted edges, not just a count of neighbors. Epsilon is a small trainable constant that controls how much a node trusts its own previous state versus its neighbors. An MLP then transforms the combined signal into the node's new representation. This weighted, signed formulation — not a generic unweighted GCN rule — is what lets the model be sensitive to connectivity strength, which matters enormously once Phase 2 starts perturbing exactly those weights.

ON SCREEN

Two equation cards reveal in sequence: the weighted-sum message rule, then the (1+ε) self-loop MLP update rule.

---

14 Pooling Into a 128-Dimensional Embedding ~26s

After two layers of message passing, every node has its own updated representation. Global mean pooling then averages all of those node vectors into one single vector describing the whole subgraph — and a final linear layer projects that down to exactly 128 dimensions. The subgraph — nodes, edges, weights, structure — has become one point in a 128-dimensional space.

ON SCREEN

A 7-node subgraph pools into a 128-cell colored vector strip; sample values shown; labeled 'linear projection → 128-D'.

---

15 One Vector Per Window: A New Kind of Sequence ~29s

Do this once for every time window in a subject's sequence, and the graph problem quietly becomes a completely different problem. G1 becomes z1. G2 becomes z2. And so on, through zT. The spatial structure of the brain has been compressed away — what is left is a sequence of vectors, evolving through time. That sequence is exactly what a recurrent model can work with.

ON SCREEN

Five card chips (G1→z1 … GT→zT) appear left to right with connecting arrows; closing line reframes it as a sequence problem.

ACT 4 — PHASE 1

16 GRU: Predicting the Next Embedding ~36s

A GRU — a gated recurrent unit — reads embeddings one at a time and keeps a hidden state summarizing everything seen so far. Feed it z1: it updates its hidden state. Feed it z2: updates again. At every step, its actual job is to predict what the next embedding will be, before being shown it. So from z1 through zt, the GRU produces a prediction — z-hat, t plus one — its best guess at what the subgraph will look like next.

ON SCREEN

Chain of z-circles feeding into GRU boxes with dashed hidden-state arrows; final card reveals the predicted z-hat(T+1).

---

17 Training: Control Subjects Only ~35s

Now watch the full Phase 1 training loop. Only the 328 train-control subjects enter. ASD subjects, validation subjects, test subjects — all blocked, visibly, from this stage. Each control subject's subgraph sequence passes through the GIN, becomes embeddings, passes through the GRU, produces a next-embedding prediction, and that prediction is compared against the real next embedding to compute a loss. That loss backpropagates through both the GIN and the GRU together — they are trained jointly, from scratch, as one system.

ON SCREEN

Three red-crossed cards (ASD / validation / test, blocked) beside a 7-step vertical pipeline ending at 'GIN+GRU updated jointly'.

---

18 Guarding Against Collapse ~35s

There is a trivial way for this loss to look good while learning nothing: map every subgraph to the same output vector. If embeddings collapse to a point, next-embedding prediction becomes free — and the model has learned literally nothing about brain dynamics. This is checked, not assumed. A variance penalty during training pushes embeddings apart, and directly after training, the measured embedding standard deviation was 1.03 to 1.16 across dimensions — far from the collapsed value of zero.

ON SCREEN

Two point-cloud scatters: a collapsed cluster (red, std≈0) vs. a spread cloud (teal, std=1.03–1.16), each in a dashed ring.

19 The Anomaly Score: A Distance Between Two Vectors ~34s

Once trained, the model is shown a real transition and asked to predict it — without being told the answer. Here is the actual next embedding. Here is the GRU's prediction. The anomaly score is simply the distance between them: the norm of their difference. A small distance means the transition looked exactly like what a healthy subject would do. A large distance means something unexpected happened — whether or not that subject turns out to be ASD.

ON SCREEN

Two 40-cell embedding vectors (actual vs. predicted); the distance equation assembles; small/large-distance comparison cards.

20 Bug: One Threshold, 280 Rolls of the Dice ~35s

Here is a real mistake, caught by running the numbers. The first approach set one threshold per individual transition — the 95th percentile of healthy validation scores — and flagged a subject if any of their roughly 280 transitions crossed it. The result: 131 of 131 test subjects flagged. Every single one, healthy or not. Why? With 280 independent rolls of the dice at a 95th-percentile cutoff, about 14 are expected to cross the line by chance alone, for any subject.

ON SCREEN

A 60-tick transition timeline with one random crossing highlighted; '131/131 flagged' in red; fix pipeline to τ_subject, 4/131.

21 Phase 1 Result: The Honest Floor ~40s

Measured on the 131 held-out test subjects: Precision 0.500, Recall 0.033, F1 0.062, AUROC 0.453, AUPRC 0.467. An AUROC of 0.453 is at chance — 0.5 would be a coin flip. This is reported as measured. ABIDE gives a whole-subject diagnosis, not a label on any specific transition, so a 'worst moment' score is a genuinely weak proxy. Phase 1's honest role in this project is to be the floor: it establishes normal dynamics, it produces the anomaly score, and it hands both to Phase 2. It is not the final result.

ON SCREEN

Five-bar chart (Precision/Recall/F1/AUROC/AUPRC) grows in with a dashed chance line at 0.5; closing hand-off pipeline to Phase 2.

ACT 5 — PHASE 2

22 From “What Is Normal” to “Can We Create a Deviation” ~24s

Phase 1 answered: what does normal look like. Phase 2 asks the harder question: starting from that same trained model, can we deliberately create a meaningful deviation — one the model itself would recognize as unusual? The trained GIN and GRU do not get retrained here. They become the engine that drives anomaly generation.

ON SCREEN

Two cards ('what does normal look like' → 'can we create a deviation') connected by a violet arrow.

---

23 Which Edges Would Fool the Model Most? ~31s

Take a real transition from a real, normal train-control subject. The model can tell us, via its own gradient, exactly how much changing each edge weight would increase its own prediction error. Every edge gets a sensitivity score. Sort them, and a clear ranking appears — most edges barely matter, a handful matter enormously. The top 15 percent of undirected edge-pairs, by that combined sensitivity, are selected as the targets.

ON SCREEN

26 edge-sensitivity bars sorted low→high; the top 15% recolor violet as the selected, most-sensitive edges.

---

24 Bug: One Big Step Overshoots ~36s

The first implementation pushed those edges with one large gradient step, sized directly to the desired severity. Tested at severities 0.25, 0.55, and 0.9, the anomaly score went 5.28, then 8.03, then dropped to 5.03. Not monotonic. A large step overshoots the region where the gradient's straight-line direction stays valid — the GIN's ReLU nonlinearities mean a big jump can land in a completely different regime. The fix: five small steps, recomputing the gradient fresh at every step — a PGD-style iterative update.

ON SCREEN

A red overshoot line chart (5.28→8.03→5.03) beside five green step-chips labeled 'step 1…5'; monotonic sequence prints below.

---

25 Validity Check: What Actually Changed ~33s

After the five perturbation steps, the generated subgraph is validated against every constraint the project cares about. Weights stay in the valid range. Symmetry is preserved — an edge's weight in both directions still matches. No self-loops are introduced. And critically: no node was added or removed, and no edge was added or removed. Only the weights of edges that already existed were allowed to move. This is a connectivity-strength anomaly, not a topology-changing one.

ON SCREEN

A five-item checklist reveals with green checks; a 'VALID GENERATED ANOMALY' card explains same-nodes/same-edges, weights only.

---

26 Structure-Aware vs. Random: A Fair Comparison ~34s

To know whether targeting matters, the identical edit budget was also spent by choosing edges at random and pushing them with a random sign — no gradient involved. Same number of edges touched, same maximum step size, only the selection method differs. Watch the severity climb on both sides. Structure-aware rises cleanly: 4.02, 5.82, 7.63. Random stays essentially flat and noisy: 2.91, 2.88, 2.84 — it never learns to get more severe, because it isn't aimed at anything.

ON SCREEN

Split screen bar charts: structure-aware severity bars climbing (4.02→5.82→7.63) vs. random bars staying flat (2.91/2.88/2.84).

---

27 The 9× Result ~39s

Here is the headline number. At high severity, structure-aware anomalies are flagged by Phase 1's own detector 45.8% of the time. Random perturbation of the exact same edit budget: 4.8%. That is roughly nine times more detectable — and it required no threshold tuning to appear. This is the central, validated evidence that the generator is exploiting real, model-relevant structure, not simply injecting bigger numbers. It does not mean these are clinically realistic ASD patterns — it means they are meaningfully different from noise, under this project's own detector.

ON SCREEN

Two big-number cards (45.8% vs. 4.8%) followed by a '≈ 9× more detectable' reveal and a non-clinical-realism disclaimer line.

---

28 7,200 Generated Records, Validated at Scale ~29s

This generation process ran across 60 base subjects, 20 seed ROIs, three severities, two methods — producing 7,200 generated anomaly records in total. Every one of them was checked against the validity constraints from before. Zero constraint violations, across all 7,200. And diversity: the mean pairwise Jaccard distance between different generated anomalies' touched-edge sets was 0.999 to 1.000 — the generator essentially never produces near-duplicate anomalies.

ON SCREEN

480-dot grid recolors violet as '7,200 generated records'; a one-item checklist (0 violations) plus a diversity Jaccard note.

---

29 The One Sobering Number ~32s

Here is what Phase 2 does not solve. Real ASD test transitions are detected at essentially the same rate as real normal transitions — 4.9% versus 5.0%. Consistent with Phase 1's near-chance AUROC. A severe synthetic anomaly is, in this sense, easier to catch than an actual clinical difference. Real clinical signal is far subtler than even a deliberately severe synthetic perturbation. This is exactly the open question Phase 3 was built to investigate.

ON SCREEN

Two-bar comparison (real normal 0.050 vs. real ASD 0.049), nearly identical; synthetic-severe-vs-real-subtle contrast lines.

ACT 6 — PHASE 3

30 Few-Shot: When Only a Handful of Real Examples Exist ~21s

Phase 3 is the payoff experiment. In practice, real labeled ASD examples for a specific task like this are scarce. So the question becomes: with only 1, or 5, or 10 real ASD support subjects, can adding Phase 2's generated anomalies actually make a detector more useful?

ON SCREEN

Three amber cards count up 1 / 5 / 10 real ASD examples; a violet line introduces generated anomalies expanding the support set.

31 Bug: A One-Number Feature Can't Change Its Own Ranking ~38s

The first version scored every subject with Phase 1's single scalar — the worst error across everything. But a logistic regression fit on one number is a strictly monotonic function of that number: no matter how it's trained, it can only ever rank subjects in the same order as the raw score itself. Result: AUROC and AUPRC came out bit-for-bit identical across every condition. The fix: a 20-dimensional feature, one score per seed ROI — now the classifier's learned weights can actually change with the training data.

ON SCREEN

Two cards contrast a rank-invariant one-scalar feature (red) with a reveal of a 20-cell per-seed vector feature (green).

32 Three Conditions, Identical Test Set ~32s

Three conditions are compared, all on the exact same 131 held-out test subjects, for the exact same support-set draw. Condition A: few-shot only — k real ASD subjects against all 328 train-control negatives. Condition B: A's positives, plus 30 structure-aware generated anomalies at high severity. Condition C: A's positives, plus 30 random-perturbation generated anomalies — the ablation. Each is repeated for k equals 1, 5, and 10, with five independent draws per shot count.

ON SCREEN

Three condition cards (A few-shot-only / B +structure-aware / C +random) with their compositions; k∈{1,5,10} × 5 draws note.

33 1-Shot: From Collapse to Usable ~36s

Look at the 1-shot condition first — the hardest setting, one real ASD example. Few-shot-only nearly collapses: F1 of 0.006. A single positive example gives logistic regression almost nothing to calibrate a decision boundary from. Add 30 generated anomalies of either kind, and F1 jumps to roughly 0.39 to 0.40 — recall goes from 0.003, essentially never finding a real ASD subject, to about 0.35, finding around one in three. This is a real, reproducible effect, with tight error bars across five trials.

ON SCREEN

1-shot F1 bar chart: few-shot-only near zero (0.006) jumping to ~0.39–0.40 for both generated conditions; recall callout below.

---

34 5-Shot and 10-Shot: The Pattern Holds ~21s

The same pattern shows up at 5-shot and 10-shot, smaller in magnitude because few-shot-only already has more to work with. At 5-shot, F1 rises from 0.149 to roughly 0.39–0.40. At 10-shot, from 0.311 to roughly 0.40–0.43. In every case, adding generated data — either kind — helps.

ON SCREEN

Two more F1 bar charts, 5-shot and 10-shot, showing the same lift pattern at smaller magnitude; closing 'F1 goes up' line.

---

35 But AUROC and AUPRC Stay Near Chance ~33s

Now the nuance that matters most. Across every single condition and every shot count, AUROC and AUPRC stay between 0.48 and 0.53 — near chance. The F1 gain came from giving the classifier a usable decision boundary, not from making the underlying feature space more separable. And structure-aware generation does not show a clear edge over random generation in these ranking metrics — even though Phase 2's own raw detectability check showed a clear nine-times advantage.

ON SCREEN

A three-series line chart (AUROC across 1/5/10-shot) hugging a dashed 0.5 chance line for all three conditions; honest-nuance card.

---

36 What the Results Actually Mean ~40s

Put together honestly: Phase 1 learns normal dynamics and gives a near-chance baseline on real ASD separation. Phase 2 generates model-relevant synthetic anomalies with strong, controllable, nine-times detectability over random. Phase 3 shows that adding generated data substantially improves F1 and recall in the data-starved few-shot setting — but AUROC and AUPRC remain near chance, and structure-aware does not yet show a clear edge over random in this classifier. The project demonstrates a promising anomaly-generation mechanism. It does not establish that synthetic anomalies improve the underlying separability of real ASD from control.

ON SCREEN

Three-stage pipeline (Phase 1 → Phase 2 → Phase 3) with sub-labels, ending in a bordered card stating the honest overall reading.

ACT 7 — FINALE

37 The Complete Pipeline ~29s

Start to finish. ABIDE the First: 871 subjects. 360 ROI time series. 30-TR sliding windows. Signed Pearson functional connectivity. Top-20% sparse weighted graphs. 20 stable local subgraphs. A weighted GIN, producing 128-dimensional embeddings. A GRU, learning normal temporal prediction. A Phase 1 anomaly score. Phase 2's gradient-guided anomaly generation, producing 7,200 validated synthetic anomalies. And Phase 3's few-shot evaluation, testing whether any of it actually helps.

ON SCREEN

12 numbered pipeline cards fill a 3×4 grid in sequence, from 'ABIDE I — 871 subjects' through 'Phase 3 few-shot evaluation'.

---

38 The Central Idea ~17s

Learn what normal brain-subgraph dynamics look like. Use that learned structure to create controlled, meaningful deviations. Then test whether those generated examples can help when real abnormal data are scarce. That is GRAPH_ANOMALY — in one sentence.

ON SCREEN

Brain with ROI dots re-forms; three headline statements fade in in sequence; closing disclaimer card on synthetic anomalies.